

Salon Hup

Project Documentation

ACTIVE DEVELOPMENT

Product and technical specification

Version 1.1 | 26 September 2026

SALON HUP

Project: Salon Hup

Backend version: 0.0.1 SNAPSHOT

Document version: 1.1

Last updated: 26 September 2026

Status: Active development

Document purpose

This document is the maintained source of truth for the Salon Hup backend. It explains the product vision, implemented business rules, application structure, data model, API surface, security model, current delivery status, known risks, and planned development. It is intended for developers, reviewers, future contributors, and stakeholders preparing the product for deployment.

The document distinguishes between implemented behavior and planned behavior. A capability is only marked as implemented when corresponding source code exists in the repository. Future changes should update this source after the implementation has been reviewed and approved.

Executive overview

Salon Hup is a platform for managing salons, salon ownership, employment, hiring, and eventually customer booking. The current backend establishes the identity, salon onboarding, administration, and workforce-management foundation needed before appointment scheduling and production deployment can be completed.

The application currently supports user registration and JWT login, salon creation requests with administrative approval, salon profile management, employment requests and owner invitations, employee leave and salon transfers, user administration, hiring-post creation, ranked applications, and owner acceptance and rejection.

Hiring-post discovery and management are the next active features. Customer-facing booking, salon services, staff availability, appointment scheduling, notifications, reviews, and payments are not yet implemented.

Product goals

- Give users a secure account and a clear role in the platform.
- Allow eligible users to request ownership of a salon profile.
- Require administrative review before a salon becomes active.
- Help salon owners recruit and manage employees.
- Allow users to request employment or respond to salon invitations.
- Preserve employment state when an employee transfers between salons.
- Publish structured hiring posts and route applications to the correct salon owner.
- Build a reliable foundation for customer booking and deployment.

Roles and responsibilities

Role	Purpose	Current permissions
USER	Default registered account and potential employee	Sign in, request salon creation, request employment, respond to invitations, and apply to hiring posts when the application workflow is completed
OWNER	User approved as the owner of a salon	Manage the owned salon, invite users, review employment requests, accept leave requests, and create hiring posts
Employee	User currently assigned to a salon	Participate in employment and leave workflows and transfer to another salon after the required workflow
ADMIN	Platform administrator	Sign in to the dashboard, review salon requests, accept or reject salon requests, and block users

Role names currently follow the source enum exactly: `USER`, `OWNER`, `ADMIN`, and `Employee`. A future normalization may standardize the casing, but that change requires migration planning.

Technology stack

Layer	Technology
Language	Java 17
Framework	Spring Boot 4.1.0
HTTP API	Spring Web MVC
Security	Spring Security, JWT, BCrypt
Persistence	Spring Data JPA and Hibernate
Database	MySQL
Validation	Spring Validation plus service-level business validation
Email foundation	Spring Mail with SMTP configuration
Build	Maven
Utilities	Lombok

Application architecture

The backend uses a layered Spring architecture.

```
Client
-> Controller
  -> Service interface
    -> Service implementation
      -> Repository
        -> MySQL database
```

Controllers define HTTP routes and receive authenticated identity information. DTOs define API input and output contracts. Service implementations contain validation and business rules. Repositories provide database access through Spring Data JPA. Entities represent persisted domain state.

The main Java package is `com.salon.backend` and contains:

- `Config` for Spring Security, password hashing, and JWT filtering.
- `Controllers` for authentication, dashboard, salons, employment, hiring posts, and users.
- `DTOs` for request and response contracts.

- **Entities** for users, salons, employment, leave requests, and hiring posts.
- **Repositories** for JPA database access.
- **Services** for authentication, administration, salon management, employment, and hiring-post rules.

All service responses use the generic `ApiResponse<T>` envelope containing `success`, `message`, and `data`.

Core domain model

User

A user stores identity and account information: first name, last name, email, username, BCrypt password hash, phone number, role, status, and an optional salon assignment.

User status values are `Active`, `Blocked`, and `Deleted`.

Salon

A salon stores its name, salon phone, owner phone, email, location, description, owner, employees, current and maximum employee counts, creation date, opening and closing times, profile picture URL, approval status, and opening status.

Salon approval status values are `Inactive`, `Requested`, `Accepted`, `Rejected`, and `DELETED`.

Employment request

An employment request links a salon, sender, receiver, optional hiring post, status, request type, source, creation time, and optional rejection reason.

Request types are `Owner_Invite` and `User_Request`. Sources are `SALON_PROFILE` and `HIRING_POST`.

Employment request statuses are `Requested`, `Accepted_Pending_Leave`, `Accepted`, `Rejected`, and `Cancelled`.

Leave request

A leave request records the employee, current salon, optional target salon, optional employment request, status, reason, and creation time. It supports both a normal departure and a transfer connected to an accepted employment request.

Leave request statuses are `Requested`, `Accepted`, and `Cancelled`.

Hiring post

A hiring post belongs to one salon and stores:

- Title and detailed description.
- Required skills in a related collection table.
- Minimum and maximum applicant age.
- Required employee location.
- Minimum years of experience.
- Number of available positions.
- `Open`, `Closed`, or `Expired` status.
- Creation and expiration timestamps.

The backend obtains salon and owner details from the authenticated owner and database. The client cannot choose another owner or salon while creating a post.

Hiring application

A hiring application links one applicant, hiring post, and employment request. It stores the applicant's date of birth, current location, years of experience, skills, optional résumé URL, optional portfolio URL, application note, and creation time.

Known identity fields including name, username, email, phone number, role, and user ID come from the authenticated user and are not accepted from the client. The linked employment request provides the application status and allows acceptance to reuse the existing employment and transfer workflows.

Implemented business workflows

Registration and login

Registration validates names, username format, supported email domains, Jordanian phone formats, password strength, and uniqueness of username, email, and phone number. New accounts receive the **USER** role and **Active** status. Passwords are stored as BCrypt hashes.

Login verifies account status and password, then returns a bearer JWT valid for 24 hours. Logout blacklists the presented token in application memory until its expiration time.

Salon onboarding

1. An authenticated user submits salon information.
2. The service validates the salon name, Jordanian city, description, email, phone number, opening hours, and employee capacity.
3. The salon is created with **Requested** status.
4. An administrator reviews the request.
5. Acceptance changes the salon to **Accepted** and promotes its user to **OWNER**.
6. Rejection changes the salon to **Rejected**.

Only the salon owner can update the owned salon through the service ownership check.

Direct employment request

A **USER** may request to join an accepted salon with available capacity. The system prevents duplicate pending requests for the same user and salon. The salon owner becomes the receiver of the request.

When the owner accepts and the user has no current salon, the user is assigned to the salon, the role becomes **Employee**, the salon employee count increases, and the request becomes **Accepted**.

Owner invitation

The owner of an accepted salon may invite another user when the salon has capacity. The system prevents self-invitations and duplicate pending invitations or requests. The invited user may accept or reject the invitation, and the owner may cancel an invitation that the owner sent.

Leave and transfer

An employee can create a leave request for the current salon. The current salon owner accepts the request. For a normal leave, the employee is removed from the salon, the role returns to **USER**, and the salon employee count decreases.

If a user who already works at a salon accepts another employment opportunity, the employment request becomes **Accepted_Pending_Leave** and a linked leave request is created. Approval by the current salon owner removes the

employee from the current salon and adds the employee to the target salon in one transaction.

Hiring-post creation

Only an authenticated **OWNER** can create a hiring post. The owner must have an associated salon with **Accepted** status. The service validates all owner-entered requirements, removes blank and duplicate skills, assigns the current time, sets the status to **Open**, and stores the post for the owner's salon.

The response automatically includes the salon ID, salon name, salon location, owner ID, and owner name from persisted data.

Hiring application and ranking

Any authenticated **USER** or **Employee** can apply to an open, unexpired hiring post unless already employed by that salon or already registered as an applicant for that post. Applications are never rejected because the applicant does not match the post requirements.

The owner receives every application ordered by match level and score:

1. **SUGGESTED** applications meet every requirement, exceed the experience requirement, and provide additional skills.
2. **MATCHED** applications meet every required skill, experience, age, and location condition.
3. **MISSING_REQUIREMENTS** applications remain visible and include exact missing skills and unmet conditions.

The match score is transparent and reproducible. Required skills contribute up to 50 points, experience up to 25, age 10, location 10, additional skills up to 5, and an optional résumé adds a 5-point bonus. The final score is capped at 100. The résumé is encouraged but may be omitted or stored as **null** without preventing submission.

Hiring application decision

Only the owner of the salon connected to a hiring post can accept or reject its applications. Rejection requires a reason and reuses the existing employment-request rejection workflow.

Acceptance reuses the employment-request acceptance workflow. An applicant without a current salon joins immediately. An applicant already employed elsewhere enters **Accepted_Pending_Leave**, and the existing transfer leave request is created. Acceptance reserves one advertised position. A post closes when no positions remain. If a pending transfer is cancelled, the reserved position is restored and an unexpired post reopens when appropriate.

Hiring-post creation contract

Owner-provided fields

Field	Rule
title	Required, at least 3 characters
description	Required, at least 10 characters
requiredSkills	At least one nonblank skill; duplicate values are removed
minimumAge	Required and at least 18
maximumAge	Required and not lower than minimum age
employeeLocation	Required
minimumYearsOfExperience	Required and zero or greater
numOfPositions	Required and greater than zero

Field	Rule
expiresAt	Required and later than the current time

Automatically generated fields

Field	Source
salon	Loaded using the authenticated owner's email
owner identity	Loaded from the authenticated account and salon relationship
status	Set to Open
createdAt	Set by the backend at creation time

Hiring application contract

Applicant-provided fields

Field	Rule
dateOfBirth	Required, valid past date, applicant must be at least 18
currentLocation	Required
yearsOfExperience	Required and zero or greater
skills	At least one nonblank skill; duplicate values are removed case-insensitively
resumeUrl	Optional; adds a ranking bonus when present
portfolioUrl	Optional
applicationNote	Optional, maximum 2000 characters

Ranking output

Field	Purpose
matchLevel	Classifies the application as SUGGESTED , MATCHED , or MISSING_REQUIREMENTS
matchScore	Orders applications within each level from highest to lowest
matchedSkills	Required skills supplied by the applicant
additionalSkills	Applicant skills beyond the post requirements
missingSkills	Required skills absent from the application
missingRequirements	Human-readable explanation of every unmet condition
resumeUploaded	Shows whether the optional résumé bonus applies

API catalog

All routes except explicitly public authentication routes require a valid bearer token under the current security configuration.

Authentication

Method	Route	Purpose
POST	/api/auth/register	Create a user account; spelling currently follows the source route
POST	/api/auth/login	Authenticate and receive a JWT
POST	/api/auth/logout	Blacklist the current JWT

Salon management

Method	Route	Purpose
GET	/api/salons	Return salons
GET	/api/salons/active	Return accepted salons
POST	/api/salons/create	Request creation of a salon
PUT	/api/salons/{salonId}/edit	Update an owned salon

Employment and leave

Method	Route	Purpose
POST	/api/employment/join	Request employment at a salon
POST	/api/employment/invite/{salonId}	Invite a user to a salon
POST	/api/employment/accept	Accept an employment request or invitation
POST	/api/employment/reject	Reject a request or invitation
POST	/api/employment/cancel	Cancel a sent employment request
POST	/api/employment/leave/request	Request departure from the current salon
POST	/api/employment/leave/accept/{requestId}	Accept a leave request
POST	/api/employment/leave/cancel/{requestId}	Cancel a leave request
POST	/api/employment/invitation/accept-and-leave	Accept an invitation and start a transfer

Hiring

Method	Route	Purpose	Status
POST	/api/hiring-posts	Create a hiring post for the authenticated owner's salon	Implemented
GET	/api/hiring-posts	Browse open hiring posts	Planned
POST	/api/hiring-posts/{id}/applications	Apply to a hiring post	Implemented
GET	/api/hiring-posts/{id}/applications	Owner reviews ranked post applications	Implemented
POST	/api/hiring-posts/applications/{id}/accept	Accept an applicant and execute employment rules	Implemented

Method	Route	Purpose	Status
POST	/api/hiring-posts/applications/{id}/reject	Reject an applicant with a reason	Implemented

Administration and users

Method	Route	Purpose
POST	/api/dashboard/login	Authenticate an administrator
GET	/api/dashboard/salons/requested	Review requested salons
POST	/api/dashboard/salons/requested/accept	Accept a salon request
POST	/api/dashboard/salons/requested/reject	Reject a salon request
POST	/api/dashboard/users/{userId}/block	Block a user
GET	/api/users	Return users
GET	/api/users/active	Return active users
GET	/api/users/blocked	Return blocked users

Security model

Spring Security runs without server sessions. The JWT filter reads the bearer token, validates its signature and expiration, loads the user by email, and places the authenticated identity and role in the security context. Passwords use BCrypt.

Public routes include login, registration, password-recovery placeholders, dashboard login, API documentation routes, and selected search routes. Dashboard salon and user routes require the **ADMIN** role. Other routes require authentication unless explicitly allowed.

Production deployment requires moving database, email, and JWT secrets to environment configuration and rotating any credentials previously stored in source configuration.

Current implementation status

Area	Status	Notes
Registration and login	Implemented	Core validation, BCrypt, and JWT available
Logout	Implemented	In-memory blacklist; not durable across restarts
Salon onboarding	Implemented	User request and admin decision available
Salon profile update	Implemented	Ownership check exists
Direct employment	Implemented	Request, invite, accept, reject, and cancel available
Leave and transfer	Implemented	Normal departure and cross-salon transfer available
Hiring-post creation	Implemented	DTO, repository, service, controller, and validation compile successfully
Hiring-post discovery	Planned	Listing and filtering endpoints required

Area	Status	Notes
Hiring applications	Implemented	Applicant profile, transparent ranking, owner review, acceptance, and rejection available
Customer booking	Planned	Domain and APIs not yet created
Automated tests	Initial	Only a basic context test exists
Deployment configuration	Planned	Profiles, secrets, migrations, packaging, monitoring, and hosting remain

Known technical risks

Authorization identity

Several employment endpoints currently accept `currentUserId` as a request parameter. The authenticated user identity should instead be obtained from Spring Security to prevent one authenticated user from acting as another user.

Entity exposure

Some controllers return JPA entities directly. This can expose password hashes, create circular JSON graphs, and couple the API to the database structure. Response DTOs should be used for all public API responses.

Secret management

Database credentials, SMTP credentials, and the JWT signing key currently exist in source configuration or code. They must be rotated and supplied through environment variables or a secrets manager before deployment.

Token revocation

The logout blacklist is stored in application memory. It disappears after a restart and does not work consistently across multiple server instances. A durable revocation or refresh-token design is required for production.

Data integrity and concurrency

Employee capacity changes can race when multiple requests are accepted simultaneously. Database constraints, transactional locking, or optimistic version fields should protect capacity and employment state.

Error and validation consistency

The application returns a successful HTTP response envelope for many business errors. A centralized exception and HTTP-status strategy will make API behavior more predictable.

Schema lifecycle

Hibernate currently uses automatic schema update. Production environments should use versioned database migrations such as Flyway or Liquibase.

Development roadmap

Priority 1 Complete hiring management

- Browse and filter open hiring posts.
- Allow owners to edit, close, and inspect their posts.
- Add scheduled expiration handling.

- Let applicants list and cancel their pending applications.
- Define whether a rejected applicant may apply again to the same post.

Priority 2 Secure existing workflows

- Replace request-supplied current user IDs with authenticated identities.
- Introduce response DTOs for users, salons, employment requests, and leave requests.
- Add role checks at route and service levels.
- Correct the admin salon lookup and owner-blocking behavior.
- Correct the salon-rejection request contract.

Priority 3 Hiring workflow hardening

- Add authorization and ranking tests.
- Protect position reservation with database locking or optimistic versioning.
- Add application history and audit timestamps.
- Add managed résumé upload storage instead of relying only on a URL.

Priority 4 Booking foundation

- Define salon services, prices, durations, and staff capabilities.
- Define weekly working hours, breaks, holidays, and time off.
- Calculate available appointment slots.
- Create booking, confirmation, cancellation, and rescheduling workflows.
- Add customer booking history and salon calendars.

Priority 5 Production readiness

- Add meaningful unit, integration, authorization, and concurrency tests.
- Introduce database migrations and environment-specific profiles.
- Move and rotate secrets.
- Add structured logging, health checks, metrics, and error tracking.
- Create Docker and deployment configuration.
- Configure production CORS, TLS, backups, and recovery procedures.
- Perform final security and performance reviews.

Quality and review policy

Every feature should move through a small reviewable slice:

1. Confirm business rules with the product owner.
2. Add or update entities only when persistence changes are required.
3. Create request and response DTOs.
4. Add repository methods with clear names.
5. Define the service interface and implement business validation.

6. Add a thin controller that uses authenticated identity.
7. Compile and run relevant tests.
8. Review the code and API behavior before moving to the next feature.
9. Update this document after the change is approved.

Documentation maintenance

The editable source is [documentation/Salon-Hup-Project-Documentation.md](#). The published PDF is [output/pdf/Salon-Hup-Project-Documentation.pdf](#).

Approved feature batches should update the document version, last-updated date, implementation-status table, relevant workflow, API catalog, domain model, and roadmap. Draft or unapproved behavior should remain marked as planned and should not be described as implemented.

Revision history

Version	Date	Summary
1.1	26 September 2026	Added ranked hiring applications, optional résumé scoring, owner decisions, position reservation, and transfer integration
1.0	26 September 2026	Initial professional project specification covering the existing backend and hiring-post creation workflow